BMAD Method vs Claude Team Skills

Side-by-Side Feature Comparison

Version 1.1 · 23 specialists · 59 skill files · 10,893 lines · MIT License

Overview

BMAD Method and Claude Team Skills (CTS) both aim to bring structured, specialist-driven workflows to AI-assisted software development. They share a common philosophy but diverge significantly in design priorities. This document compares them feature-by-feature so teams can make an informed choice.

At a Glance

Metric	BMAD Method	Claude Team Skills
Specialists / roles	~10-15 personas	23 specialists
Installation	npm package	Copy markdown — zero deps
Platform	ChatGPT, Gemini, Claude, 10+ IDEs	Claude Code native
Config format	YAML (Agent-as-Code)	Plain markdown
Total files	Varies by bundle	59 .md skill files
Total lines	Varies	10,893 lines
License	MIT	MIT

Feature-by-Feature Comparison

Feature	BMAD Method	Claude Team Skills	CTS Rationale
Installation	npm installer required	**Copy markdown — zero deps**	No Node.js, no build step. Works in any editor.
IDE / platform support	10+ IDEs (VS Code, Cursor, Windsurf, Claude Code, etc.)	**Claude Code native only**	One platform done exceptionally well beats many done pass
Agent style	Named personas (characters with backstories)	**Role-based skills (function definitions)**	Role-based uses fewer tokens and is easier to maintain.
Multi-agent debate	Party Mode — agents debate simultaneously	**Sequential handoffs with written specs**	Sequential saves ~70% tokens vs loading all agents.
Bundle structure	Modular expansion packs (install what you need)	**Single download — everything included**	One bundle = no dependency hell, simpler onboarding.
Complexity control	Scale-adaptive intelligence (auto-detects)	**YAGNI gates — user explicitly passes complexity**	Explicit control prevents over-engineering by default.
Config format	YAML compilation (Agent-as-Code)	**Plain markdown — editable in any text editor**	Markdown is universal, requires no build step.
Session memory	Context within session	**8 append-only .logs/ files — persistent across sessions**	Decisions and lessons survive session resets.
Workflow phases	Task-based (flexible)	**6 mandatory phases with hard gates**	Non-negotiable gates prevent code shipping before VERIFY
Coverage gate	Not enforced by framework	**>=80% combined unit + integration coverage required before SHIP**	Coverage is a quality floor, not a suggestion.
CI monitoring	Not built-in	**DevOps watches every push — CI red = top priority**	CI red is a blocker, not a background event.
Security integration	Optional security persona	**Security Engineer engaged at DESIGN, plan review, and VERIFY phases**	Security at design time is cheaper than security at ship time
Token efficiency	Varies by persona count loaded	**~3-5K for routing (1 specialist at a time)**	vs ~35K+ if all 23 skills loaded simultaneously.

Feature	BMAD Method	Claude Team Skills	CTS Rationale
KPI tracking	Not built-in	**10 KPIs tracked across every sprint**	Data-driven decisions require persistent metrics.
Document-first	Not enforced	**PRD -> Architecture -> Stories enforced**	Prevents code-first rework anti-pattern.
Adversarial review	Not built-in	**Test Architect runs adversarial review**	SQLi, XSS, IDOR, CSRF tested systematically. Edge conditions tested systematic
Creative/marketing	Not included	**5 creative specialists: Digital Marketer, Copywriter, Content Marketer, Creative Intelligence**	Full-stack product team, not just engineering.
Design quality loop	Not built-in	**Motion Designer + Design Loop**	One-shot design; the loop iterates until it passes fresh-context critics.

The 23 CTS Specialists

#	Specialist	Group	Unique Capabilities vs Generic AI
1	**Orchestrator**	Routing	Routes every request; enforces 6 phases, handoffs, coverage gate
2	**Project Manager**	Management	Charter, WBS, risk matrix, PERT, PRD, Green/Amber/Red status
3	**Scrum Master**	Management	MoSCoW backlog, Fibonacci points, Given/When/Then stories
4	**Tech Lead**	Management	ADRs, YAGNI gate, SOLID/DRY/KISS, debt triage
5	**System Designer**	Architecture	NFR capture, capacity planning, topology, SLA/SLO, SDRs
6	**Software Architect**	Architecture	Clean/Hexagonal/DDD, CQRS, fitness functions, dependency rules
7	**UX Designer**	Design	3-tier flows, wireframe states, personas, Nielsen heuristics
8	**UI Designer**	Design	Design tokens, 4px grid, 12-col layout, anti-slop aesthetic
9	**Motion Designer**	Design	Motion tokens, micro-interactions, scroll/transitions, React→MP4, generative BGs
10	**Design Loop**	Design	Independent fresh-context critics (brief/system/craft) instead of self-review
11	**Security Engineer**	Security	STRIDE model, auth decision tree, SOC2/HIPAA/PCI-DSS, IR template
12	**DBA**	Data	PostgreSQL-default, 4 index types, EXPLAIN ANALYZE, backup strategy
13	**Backend Dev**	Dev	RESTful checklist, Route→Controller→Service→Repository pattern
14	**Frontend Dev**	Dev	State management decision table, a11y checklist, breakpoints
15	**Tester**	Quality	Testing pyramid, AAA, >=80% coverage gate (non-negotiable)
16	**Test Architect**	Quality	Risk scoring, ATDD, adversarial review, edge case hunter
17	**Deployment**	Operations	Rolling/Blue-Green/Canary procedures, pre/post checklists
18	**DevOps/DevSecOps**	Operations	Semgrep/Trivy/Checkov/Gitleaks, hardened Dockerfile, CI monitoring
19	**Creative Intelligence**	Creative	7 ideation frameworks (SCAMPER, Design Thinking, First Principles)
20	**Digital Marketer**	Marketing	SEO checklist, funnel, GA4 events, A/B testing rules
21	**Copywriter**	Marketing	AIDA/PAS/BAB, 5 headline types, 8-section landing page
22	**Content Marketer**	Marketing	Pillars, calendar, repurposing chain blog→LinkedIn→email→video
23	**Project Monitor**	Memory	8 append-only logs, 10 KPIs, status reports, session resumption

Why CTS Skips These BMAD Features

CTS intentionally excludes several BMAD features. These are design choices, not gaps. Each exclusion serves the core principle: one platform, done exceptionally well, with the minimum complexity needed.

Skipped Feature	BMAD Has	CTS Position
NPM installer	✓ Required	Zero-dependency is non-negotiable. Copy markdown, start working.
10+ IDE support	✓ Available	Claude Code native — deeper integration beats broader coverage.
Named agent personas	✓ Included	Role-based skills use fewer tokens and are easier to audit.
Party Mode debate	✓ Included	Sequential handoffs save ~70% tokens with no quality loss.
Expansion packs	✓ Available	Single bundle — one download, no dependency management.
Auto complexity scale	✓ Included	Explicit YAGNI gates — user chooses, AI doesn't assume.
YAML Agent-as-Code	✓ Included	Plain markdown is universal, editable, versionable by anyone.
ChatGPT / Gemini web	✓ Available	Claude Code native — one tool, maximum depth.